

STM32F407 裸机工程手册

——基于 CMake + arm-none-eabi-gcc 的构建、目录结构与使用说明

项目：MAKE

2026 年 8 月

目录

1 项目简介

本项目是一个 **STM32F407** 系列微控制器的裸机（Bare-metal）工程，使用 **CMake** 组织构建，配合 **arm-none-eabi-gcc** 交叉工具链，采用 ST 官方较老的标准外设库（**StdPeriph Driver**）而非 HAL 库。

1.1 硬件目标

- 芯片：STM32F407VET6
- 内核：ARM Cortex-M4（带 FPU），主频最高 168 MHz
- Flash：512 KB；RAM：192 KB（含 64 KB CCM）
- 当前演示功能：GPIO 点亮/翻转 PD12 上的 LED（跑马灯）

1.2 软件栈

组件	说明
构建系统	CMake (≥ 3.24), Unix Makefiles / Ninja / MinGW
交叉编译器	arm-none-eabi-gcc（本项目用 GNU 16.2.0）
库	ST 标准外设库 StdPeriph + ARM CMSIS
链接脚本	linker/STM32F407VET6.ld
启动文件	startup/startup_stm32f40_41xxx.s
文档	本文档（LaTeX, ctex + xelatex）

1.3 工程特点

- 纯 C + 汇编，不依赖任何 IDE 工程文件（无 Keil/IAR 工程）。
- 使用 `-nostdlib -nostartfiles`，完全不链接标准 C 库，启动流程由自己的 `startup` 文件完成。
- 工具链文件（`cmake/arm-none-eabi-gcc.cmake`）与构建脚本解耦，跨平台（macOS / Linux / Windows）复用同一份 CMake。
- 应用代码（`User/`）与 ST 设备支持代码（`Core/`）、厂商库（`Driver/`）分层清晰。

1.4 为什么用 CMake 而不是 IDE

传统 STM32 开发通常依赖 Keil MDK / IAR / STM32CubeIDE 等 IDE，优点是开箱即用，缺点是工程文件与 IDE 绑定、难以做持续集成（CI）、难以在命令行自动化构建。使用 CMake 的好处是：

1. 一份 `CMakeLists.txt` 跨三平台复用，无需手工维护多个工程文件；
2. 构建过程可脚本化，方便接入 CI / 自动化测试；

3. 依赖关系显式、可追溯，新增/删除源文件只需改一处；
4. 不绑定具体编辑器，VSCode / CLion / Vim 均可配合使用。

2 目录结构及使用

本节说明工程顶层目录的职责，以及日常开发时应该在哪里增删文件。

2.1 整体目录树

```

1  MAKE/
2    CMakeLists.txt           # 顶层构建脚本（源文件、头文件、编译/链接选项）
3    CMakePresets.json       # CMake 预设（目前为空，可扩展）
4    cmake/
5      arm-none-eabi-gcc.cmake # 交叉编译工具链文件
6    User/                    # 应用层：你的业务代码
7      src/
8        main.c               # 应用入口（LED 演示）
9      inc/
10     stm32f4xx_conf.h       # 外设库配置（启用哪些外设）
11   Core/                    # 设备支持层：ST 提供的系统代码
12     src/
13       system_stm32f4xx.c    # 时钟初始化 SystemInit/SetSysClock
14     inc/                    # 预留：将来放设备相关的头文件
15   Driver/                  # 厂商库（第三方，原则上只读不修改）
16     CMSIS/
17       Include/              # ARM Cortex-M 内核头文件
18       Device/ST/STM32F4xx/Include/
19         stm32f4xx.h         # 设备寄存器定义
20         system_stm32f4xx.h  # 时钟函数声明
21     STM32F4xx_StdPeriph_Driver/
22       inc/                  # 外设库头文件
23       src/                  # 外设库实现
24   startup/                  # 各型号启动文件（汇编 .s）
25   linker/                   # 链接脚本（.ld）
26   build/                    # 构建产物（应加入 .gitignore）
27   latex/                    # 本文档（LaTeX）

```

2.2 各目录职责

User/ 应用层，放你自己写的代码。新增模块（如 led.c、uart.c）应放在 User/src/，对应头文件放 User/inc/。

Core/ 设备支持层，放与芯片本身强相关的系统代码，目前只有 ST 的 system_stm32f4xx.c（时钟初始化）。Core/inc/ 预留，将来若有板级/设备级头文件可放这里。

Driver/ 第三方厂商库（CMSIS + 标准外设库）。这部分是 ST 官方源码，原则上不要修改，升级库时直接整体替换。

startup/ 启动文件。每个芯片型号一个.s，本工程实际使用的是 `startup_stm32f40_41xxx.s`。

linker/ 链接脚本，决定代码/数据在 Flash 和 RAM 中的布局。

cmake/ 工具链文件，告诉 CMake 用哪套交叉编译器。

build/ 构建输出目录（中间文件、.elf/.bin/.hex/.map），由 CMake 生成，不应提交到版本库。

latex/ 项目文档（本文档）。

2.3 日常使用：新增一个源文件

假设要新增一个串口模块 `uart.c` / `uart.h`：

1. 把 `uart.c` 放到 `User/src/`，`uart.h` 放到 `User/inc/`；
2. 在 `CMakeLists.txt` 的 `add_executable` 里追加一行：

```
1 add_executable(${TARGET}
2     ${CMAKE_SOURCE_DIR}/User/src/main.c
3     ${CMAKE_SOURCE_DIR}/User/src/uart.c      # 新增
4     ...
5 )
```

头文件无需在 CMake 里逐一登记，因为 `User/inc` 已经在 `target_include_directories` 里，编译器会自动搜索。

3 本项目修改记录

本章记录本工程从 ST 模板到「可编译的 CMake 工程」过程中所做的关键修改，以及每处修改的原因。理解这些原因，比记住改了什么更重要。

3.1 背景：Homebrew 工具链没有 newlib

在 macOS 上通过 Homebrew 安装的 `arm-none-eabi-gcc`（如 `arm-none-eabi-gcc 16.2.0`）是用 `--without-headers` 编译的：它只带 `libgcc`，不带 `newlib`，因此工具链里没有 `libc.a` / `libm.a`，也没有 `nosys.specs` / `nano.specs`。

而 ST 的模板工程默认假设工具链带有完整的 `newlib`，这导致了下面前两处修改。

3.2 修改一：链接脚本去掉不存在的库

ST 模板的链接脚本里有一段 `/DISCARD/`，让链接器去打开 `libc.a`、`libm.a`、`libgcc.a` 并丢弃其内容：

```

1  /* 修改前 (ST 模板) */
2  /DISCARD/ :
3  {
4      libc.a ( * )
5      libm.a ( * )
6      libgcc.a ( * )
7  }

```

由于本工具链没有 libc.a / libm.a, 链接时报错:

```

1  arm-none-eabi-ld: cannot find libc.a: Invalid argument
2  collect2: error: ld returned 1 exit status

```

又因为本工程使用了 `-nostdlib`, 本来就不链接这些标准库, 所以这个 `/DISCARD/` 块是多余的。修改后只保留存在的 `libgcc.a`:

```

1  /* 修改后 */
2  /DISCARD/ :
3  {
4      libgcc.a ( * )
5  }

```

3.3 修改二: 启动文件去掉 `__libc_init_array`

ST 模板的启动文件在进入 `main` 之前会调用 `__libc_init_array`, 该函数负责执行 C++ 静态构造函数, 由 `newlib` 提供。纯 C 工程没有静态构造函数, 且本工具链没有 `newlib`, 因此链接时报 `undefined reference to __libc_init_array`。

```

1  /* 修改前 */
2  bl SystemInit
3  bl __libc_init_array ; 需要 newlib, 本工具链没有
4  bl main
5
6  /* 修改后 */
7  bl SystemInit
8  bl main

```

删除后启动流程依然是完整的: `Reset_Handler` → 拷贝 `.data` → 清零 `.bss` → `SystemInit()` → `main()`。

3.4 修改三: CMake 将厂商头文件标记为 `SYSTEM`

ST 的 `stm32f4xx.h` 里存在重复的宏定义 (如 `DBGMCU_APB2_FZ_DBG_TIM1_STOP` 定义了两次), GCC 16 会报 `redefined` 警告。厂商头文件的告警不应算作本工程的告警, 因此把它们作为 `SYSTEM` include (即 `-isystem`) 处理:

```

1  # 厂商头文件: SYSTEM, 忽略其内部告警
2  target_include_directories(${TARGET} SYSTEM PRIVATE

```

```

3     ${CMAKE_SOURCE_DIR}/Driver/CMSIS/Include
4     ${CMAKE_SOURCE_DIR}/Driver/CMSIS/Device/ST/STM32F4xx/Include
5     ${CMAKE_SOURCE_DIR}/Driver/STM32F4xx_StdPeriph_Driver/inc
6 )

```

这样既屏蔽了 ST 头文件自身的告警，又不会在以后升级 CMSIS 时丢失修改。

3.5 修改四：修正 system_stm32f4xx.c 的误导缩进

ST 的 system_stm32f4xx.c 里，忙等 while 后面手滑残留了一个空的 { } 块，触发 -Wmisleading-indentation 告警：

```

1  /* 修改前 */
2  while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);
3  {
4      ; 残留空块，缩进有误导性
5  }
6  /* 修改后 */
7  while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);

```

3.6 修改五：启动文件命名修正

原目录下 startup_stm32f427x.s 命名有误(少了末尾一个 x)，已更名为 startup_stm32f427xx.s，与其它型号命名保持一致。

3.7 修改六：应用/设备代码分层（详见第?? 节）

把用户代码（main.c、stm32f4xx_conf.h）从 Core/ 中拆出，独立成 User/，与 ST 设备代码 Core/ 分层。具体见下一章。

4 目录优化：User 与 Core 分离

4.1 问题：用户代码与 ST 代码混在一起

最初的工程把 main.c（你的应用代码）和 system_stm32f4xx.c（ST 的系统代码）都放在 Core/src/ 里，stm32f4xx_conf.h 也放在 Core/inc/ 里：

```

1  Core/
2      src/
3          main.c          # 用户代码（不该在这里）
4          system_stm32f4xx.c  # ST 系统代码
5      inc/
6          stm32f4xx_conf.h  # 应用配置（不该在这里）

```

这样做的坏处是：你的代码和厂商代码边界不清。当升级 CMSIS / 标准外设库（通常整个替换 Driver/ 目录）时，容易误伤自己的代码；反之，定位「我写的代码」时也要在 ST 源码里翻找。

4.2 分层后的结构

将应用层与设备支持层分开：

```
1 User/                                # 应用层：你写的代码
2   src/main.c                         # 应用入口
3   inc/stm32f4xx_conf.h              # 外设库配置
4
5 Core/                                # 设备支持层：ST 系统代码
6   src/system_stm32f4xx.c            # 时钟初始化
7   inc/                               # 预留：设备相关头文件
```

分层原则：

- **User/**：业务逻辑、外设应用封装，随项目变化频繁修改；
- **Core/**：与芯片强相关的系统代码，改动少，接近厂商源码；
- **Driver/**：纯第三方库，只读，升级时整体替换。

4.3 对应的 CMake 改动

源文件列表与头文件路径随之更新：

```
1 add_executable(${TARGET}
2   ${CMAKE_SOURCE_DIR}/User/src/main.c      # 原来在 Core/src
3   ${CMAKE_SOURCE_DIR}/Core/src/system_stm32f4xx.c
4   ...
5 )
6
7 target_include_directories(${TARGET} PRIVATE
8   ${CMAKE_SOURCE_DIR}/User/inc              # 原来在 Core/inc
9   ${CMAKE_SOURCE_DIR}/Core/inc
10 )
```

4.4 后续可进一步优化的方向

当前结构已经够用，但如果工程继续增长，可以考虑：

1. 把 **Driver** 拆成独立仓库 / **Git submodule**：厂商库体积大且与业务无关，用 submodule 引入可保持主仓库干净、升级方便。
2. 引入 **App/** 或按功能分子目录：当 **User/src/** 下文件变多时，按功能拆成 **User/src/bsp/**、**User/src/app/** 等，用 **target_sources** 分目录添加。
3. 用 CMake 的 **object library** / **interface library**：把 **Driver/**、**Core/** 各自封装成一个静态库目标，主程序只链接这些库，依赖关系更清晰、编译更可复用。
4. 用 **CMakePresets.json** 管理多套配置：例如一份 debug (**-O0 -g**) 一份 release (**-Os**)，避免命令行里反复写长参数。

5. 提取 CPU 编译选项到工具链文件或一个变量：目前 CPU_FLAGS 定义在 CMakeLists.txt 顶部，若以后支持多个芯片型号，可改成按型号选择的缓存变量。
6. 补充.gitignore：忽略 build/、*.elf/*.bin/*.hex/*.map 等产物，目前这些文件仍出现在 git status 里。

5 macOS / Linux 构建方法

5.1 环境准备

1. 安装 CMake (≥ 3.24);
2. 安装 arm-none-eabi 交叉工具链：
 - macOS (Homebrew) : `brew install arm-none-eabi-gcc` 或 `brew install --cask gcc-arm-embedded`;
 - Linux: `sudo apt install gcc-arm-none-eabi` (Debian/Ubuntu) 或使用 ARM 官方工具链包。

注意：Homebrew 的 `arm-none-eabi-gcc` 不带 `newlib` (见第?? 前文的修改记录)，因此本工程已按「无标准库」方式配置；若用 ARM 官方工具链 (带 `newlib`)，本工程同样可以正常编译。

5.2 构建步骤

在工程根目录执行：

```

1  # 1) 删除旧构建目录 (首次构建可省略)
2  rm -rf build
3
4  # 2) 用工具链文件配置, 生成 Makefile
5  cmake -S . -B build -G "Unix Makefiles" \
6    -DCMAKE_TOOLCHAIN_FILE=$PWD/cmake/arm-none-eabi-gcc.cmake
7
8  # 3) 编译
9  cmake --build build

```

5.3 产物

编译成功后，build/ 下会生成：

```

1  build/MAKE      # ELF (含调试信息)
2  build/MAKE.bin  # 裸二进制 (烧录用)
3  build/MAKE.hex  # Intel HEX (烧录用)
4  build/MAKE.map  # 链接地址映射表

```

其中 `.bin/.hex` 由 CMakeLists.txt 里的 `POST_BUILD` 步骤调用 `arm-none-eabi-objcopy` 自动生成。

5.4 重新编译增量

修改代码后只需：

```
1 cmake --build build
```

CMake 会自动重编被改动的文件并重新链接，速度很快。

5.5 更换生成器

也可以用 Ninja 加速：

```
1 cmake -S . -B build -G Ninja \  
2   -DCMAKE_TOOLCHAIN_FILE=$PWD/cmake/arm-none-eabi-gcc.cmake \  
3 cmake --build build
```

6 Windows 构建方法

本工程完全跨平台，Windows 下同样用 CMake + arm-none-eabi-gcc 构建，只是工具链安装方式和生成器（Generator）略有不同。

6.1 第一步：安装交叉工具链

三选一即可：

1. **ARM 官方工具链(推荐)**:从 <https://developer.arm.com/downloads/-/arm-gnu-toolchain-download> 下载 Windows 版的 arm-gnu-toolchain-*-mingw-w64-i686-arm-none-eabi.zip, 解压到例如 C:\tools\arm-gnu-toolchain。
2. **MSYS2(包管理,方便升级)**:安装 MSYS2 后执行 `pacman -S mingw-w64-x86_64-arm-none-eabi-gcc`。
3. **复用 STM32CubeIDE 自带工具链**:如果装了 STM32CubeIDE,其安装目录下有 STM32CubeIDE\plugins\

6.2 第二步：安装 CMake 与生成器

- 安装 CMake (<https://cmake.org/download/>)，安装时勾选「Add CMake to the system PATH」。
- 生成器二选一：
 - **MinGW Makefiles**:需要 mingw32-make.exe,可从 MSYS2 装 mingw-w64-x86_64-make,或装独立的 MinGW;
 - **Ninja**: 下载 ninja.exe 放入 PATH (推荐,速度快)。

6.3 第三步：配置 PATH

把工具链的 bin 目录加入系统 PATH，例如 C:\tools\arm-gnu-toolchain\13.2.Rel1\bin。
验证安装：

```
1 arm-none-eabi-gcc --version
2 cmake --version
3 ninja --version          # 若用 Ninja
```

6.4 第四步：构建

在工程根目录打开 PowerShell 或 cmd：

```
1 # 删除旧构建目录（可选）
2 Remove-Item -Recurse -Force build
3
4 # 用工具链文件配置（Ninja 生成器）
5 cmake -S . -B build -G Ninja `
6     -DCMAKE_TOOLCHAIN_FILE="$PWD/cmake/arm-none-eabi-gcc.cmake"
7
8 # 编译
9 cmake --build build
```

若用 MinGW Makefiles，把 -G Ninja 换成 -G "MinGW Makefiles" 即可。

6.5 Windows 与 macOS 的差异

- **newlib 是否自带**：ARM 官方 Windows 工具链是带 newlib 的（有 libc.a / libm.a / nosys.specs），因此 ST 模板原始的链接脚本 /DISCARD/ 块本可以正常使用；但本工程已经按「无标准库」方式精简（见第?? 前的修改记录），在 Windows 上同样能编译，两套配置兼容。
- **路径分隔符**：工具链文件里的编译器名 arm-none-eabi-gcc 不带 .exe 后缀，CMake 在 Windows 上会自动补 .exe 查找，无需修改。
- **换行符**：Windows 下 git 若开启 core.autocrlf，可能把 .s/.c 文件换行改为 CRLF，一般不影响编译；若出现大量无关 diff，建议在 .gitattributes 里对源码文件强制 LF。

6.6 常见问题

arm-none-eabi-gcc 不是内部或外部命令：PATH 未配置或未重开终端，重新打开终端让 PATH 生效。

CMake 找不到编译器：工具链文件里写的是相对 PATH 的名字，确认 bin 目录在 PATH 中，或在工具链文件里改成绝对路径。

mingw32-make 找不到：改用 Ninja，或安装 MinGW 的 make。

7 CMake 说明

本章逐段解释本工程的 CMake 配置，帮助理解「为什么这样写」。

7.1 工具链文件 cmake/arm-none-eabi-gcc.cmake

```

1  set(CMAKE_SYSTEM_NAME Generic)           # 目标系统：裸机，无操作系统
2  set(CMAKE_SYSTEM_PROCESSOR arm)
3  set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY) # 交叉编译时只能"试编译"，不能"试运行"
4
5  set(CMAKE_C_COMPILER      arm-none-eabi-gcc)
6  set(CMAKE_ASM_COMPILER    arm-none-eabi-gcc)
7  set(CMAKE_CXX_COMPILER    arm-none-eabi-g++)
8  set(CMAKE_OBJCOPY         arm-none-eabi-objcopy)
9  set(CMAKE_OBJDUMP         arm-none-eabi-objdump)
10 set(CMAKE_SIZE             arm-none-eabi-size)

```

要点：

- **CMAKE_SYSTEM_NAME Generic**：告诉 CMake 这是裸机/无系统目标，不要链接宿主机（macOS/Linux）的标准库。
- **CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY**：交叉编译时，CMake 的编译器探测会尝试运行一个测试程序，但 arm 程序无法在宿主机上运行；设成静态库让探测只编译不运行，否则会报 `cannot run test program while cross compiling`。
- 编译器名不带路径：依赖 PATH。若想更鲁棒，可写成绝对路径。

7.2 CMakeLists.txt 逐段解析

7.2.1 项目与语言

```

1  cmake_minimum_required(VERSION 3.24)
2  project(MAKE LANGUAGES C ASM)
3  set(CMAKE_C_STANDARD 99)
4  set(CMAKE_C_STANDARD_REQUIRED ON)

```

`LANGUAGES C ASM` 声明只用 C 和汇编（不启用 C++），ASM 语言让 CMake 能直接编译 .s 启动文件。`CMAKE_C_STANDARD` 指定 C 标准（本工程当前用 C99）。

7.2.2 CPU 选项

```

1  set(CPU_FLAGS
2      -mcpu=cortex-m4      # 目标内核
3      -mthumb              # 使用 Thumb-2 指令集（Cortex-M 只支持 Thumb）
4      -mfpu=fpv4-sp-d16    # 单精度 FPU

```

```

5     -mfloat-abi=hard          # 硬浮点 ABI (用 FPU 寄存器传参)
6 )

```

这些选项在编译和链接时都要加上 (target_compile_options 与 target_link_options 都引用了 CPU_FLAGS)，否则链接阶段会因浮点 ABI 不匹配而报错。

7.2.3 目标与源文件

```

1  add_executable(${TARGET}
2      ${CMAKE_SOURCE_DIR}/User/src/main.c
3      ${CMAKE_SOURCE_DIR}/Core/src/system_stm32f4xx.c
4      ${CMAKE_SOURCE_DIR}/startup/startup_stm32f40_41xxx.s
5      ${CMAKE_SOURCE_DIR}/Driver/STM32F4xx_StdPeriph_Driver/src/misc.c
6      ...
7 )

```

这里只显式列出了当前用到的几个外设库源文件 (GPIO、RCC、SYSCFG、MISC)。如果以后用到其它外设，把对应 stm32f4xx_*.c 加进来即可，不必把整个 Driver/src/ 全部编译 (省时间、省体积)。

7.2.4 头文件路径与 SYSTEM

```

1  # 自己的头文件：正常警告
2  target_include_directories(${TARGET} PRIVATE
3      ${CMAKE_SOURCE_DIR}/User/inc
4      ${CMAKE_SOURCE_DIR}/Core/inc
5  )
6
7  # 厂商头文件：SYSTEM，忽略其内部告警
8  target_include_directories(${TARGET} SYSTEM PRIVATE
9      ${CMAKE_SOURCE_DIR}/Driver/CMSIS/Include
10     ${CMAKE_SOURCE_DIR}/Driver/CMSIS/Device/ST/STM32F4xx/Include
11     ${CMAKE_SOURCE_DIR}/Driver/STM32F4xx_StdPeriph_Driver/inc
12 )

```

SYSTEM 关键字等价于给编译器传 -isystem，使这些目录里的头文件被视为「系统头文件」，其内部告警被抑制。这是处理第三方头文件告警的标准做法。

7.2.5 编译宏定义

```

1  target_compile_definitions(${TARGET} PRIVATE
2      STM32F40_41xxx          # 芯片型号宏，让 stm32f4xx.h 选择对应寄存器组
3      USE_STDPERIPH_DRIVER    # 让 stm32f4xx.h 包含 stm32f4xx_conf.h
4  )

```

这两个宏是 ST 头文件的「开关」：不定义 STM32F40_41xxx 会导致 stm32f4xx.h 报错 Please select the target device; 不定义 USE_STDPERIPH_DRIVER 则外设库头文件不会被包含。

7.2.6 链接选项

```

1 target_link_options(${TARGET} PRIVATE
2     ${CPU_FLAGS}
3     -T${LD_SCRIPT}          # 指定链接脚本
4     -nostdlib                # 不链接标准 C 库
5     -nostartfiles           # 不用编译器自带的启动文件
6     -Wl,--gc-sections       # 丢弃未使用的段（减体积）
7     -Wl,-Map=${CMAKE_BINARY_DIR}/${TARGET}.map # 生成 map 文件
8 )

```

- `-nostdlib / -nostartfiles`: 裸机关键选项, 启动流程完全由 `startup/*.s` 自己负责。
- `--gc-sections`: 配合编译选项 `-ffunction-sections -fdata-sections` 才能生效; 本工程目前未显式加这两个编译选项, 若想进一步瘦身, 可在 `target_compile_options` 里补上。

7.2.7 后处理: 生成 .bin/.hex

```

1 add_custom_command(TARGET ${TARGET} POST_BUILD
2     COMMAND ${CMAKE_OBJCOPY} -O binary $<TARGET_FILE:${TARGET}> ${BIN}
3     COMMAND ${CMAKE_OBJCOPY} -O ihex $<TARGET_FILE:${TARGET}> ${HEX}
4     COMMAND ${CMAKE_SIZE} $<TARGET_FILE:${TARGET}>
5     ...
6 )

```

链接完成后自动调用 `objcopy` 生成 `.bin / .hex`, 并打印各段大小 (`text/data/bss`)。

7.3 CMakePresets.json

本工程根目录下有一个 `CMakePresets.json` (目前为空)。预设的作用是把「生成器 + 工具链 + 选项」打包成一个名字, 之后一行命令即可配置:

```

1 {
2   "version": 3,
3   "configurePresets": [
4     {
5       "name": "arm",
6       "generator": "Ninja",
7       "toolchainFile": "${sourceDir}/cmake/arm-none-eabi-gcc.cmake",
8       "binaryDir": "${sourceDir}/build"
9     }
10  ]
11 }

```

配置时即可简化为 `cmake --preset arm`, 构建用 `cmake --build --preset arm`。

8 C 语言要点（嵌入式视角）

裸机开发里，C 语言的一些特性会被高频使用，理解它们有助于读懂 CMSIS 和标准外设库源码。本节挑重点讲。

8.1 固定宽度整数：stdint.h

嵌入式里寄存器宽度是硬件定死的，必须用固定宽度类型而不是 `int`：

```
1 #include <stdint.h>
2
3 uint32_t value;    // 一定是 32 位无符号
4 uint16_t reg16;    // 16 位
5 uint8_t  flag;     // 8 位
6 int32_t  signed32;
```

`int` 的宽度随平台/编译器而变，在寄存器操作里用 `int` 是隐患。

8.2 volatile：防止编译器优化掉内存访问

外设寄存器是内存映射的，硬件会异步修改它们，必须用 `volatile` 告诉编译器「每次都要真正读写内存，不能缓存、不能省略」：

```
1 volatile uint32_t *reg = (volatile uint32_t *)0x40023800;
2 *reg = 0x01;           // 必须真正写到该地址
3 while (*reg & 0x02) { /* 必须每次重新读 */ }
```

漏掉 `volatile` 的典型后果是：编译器把循环里的读取优化掉，导致忙等死循环。

8.3 CMSIS 的寄存器访问模型

CMSIS 不直接写裸指针，而是用一个结构体覆盖外设地址空间，并用 `__IO`/`__IM`/`__OM` 标注读写属性：

```
1 /* __IO = volatile 可读可写，__IM = 只读，__OM = 只写 */
2 typedef struct {
3     __IO uint32_t MODER;    /* 模式寄存器 */
4     __IO uint32_t OTYPER;
5     __IO uint32_t OSPEEDR;
6     __IO uint32_t PUPDR;
7     __IO uint32_t IDR;
8     __IO uint32_t ODR;
9 } GPIO_TypeDef;
10
11 #define GPIOD ((GPIO_TypeDef *)0x40020C00) /* 外设基地址 */
12 GPIOD->ODR |= (1U << 12);                /* 直接操作寄存器 */
```

这种写法类型安全、可读性好，是 STM32 库（HAL/StdPeriph/CMSIS）的通用套路。

8.4 位操作

```
1  #define BIT(n) (1UL << (n))
2
3  uint32_t r = 0;
4
5  r |= BIT(12);      // 置位：第 12 位 = 1
6  r &= ~BIT(12);     // 清位：第 12 位 = 0
7  r ^= BIT(12);      // 翻转
8  if (r & BIT(12)) {} // 测试
9
10 // 写多位（例如写 MODE 字段，占 2 位）
11 r = (r & ~(0x3UL << 10)) | (value << 10); // 先清空字段，再写入
```

「先清空、再写入」是操作寄存器字段的的标准范式，库里的 `GPIO_Init` 等函数内部正是这么写的。

8.5 freestanding 与 -nostdlib

标准 C 区分「宿主环境（hosted）」和「独立环境（freestanding）」。裸机没有操作系统，属于 freestanding：

- 编译选项加了 `-ffreestanding`，告诉编译器不假设有 `libc`；
- 链接选项加了 `-nostdlib -nostartfiles`，不链接标准库和启动文件；
- 因此 `printf/malloc/memcpy` 等默认不可用。需要时要么自己实现，要么链接 `newlib` 并配置 `syscalls`。

这也是本工程在链接脚本/启动文件里去掉 `libc.a` 和 `__libc_init_array` 的根本原因。

8.6 weak 弱符号与中断处理函数

启动文件里每个中断向量都指向一个弱符号的默认处理函数：

```
1  .weak WWDG_IRQHandler
2  .thumb_set WWDG_IRQHandler, Default_Handler
```

`.weak` 表示「如果你在别处定义了同名函数，就用你的；否则用默认的」。所以用户在 `main.c` 里写一个 `void TIM2_IRQHandler(void)` 就能覆盖默认实现、接管中断，无需改启动文件。

8.7 const 与 flash 布局

```
1  const char msg[] = "hello"; // 放到 .rodata (Flash)，不占 RAM
```

裸机 RAM 很小，常量应尽量用 `const` 让链接器把它们放进 Flash 的只读段，而不是可写的 `.data`。

9 嵌入式开发要点

本章从「上电到 main」的链路出发，讲解 STM32 裸机运行机制。

9.1 启动流程

上电复位后，CPU从地址 0x00000000 取栈顶指针（SP）和复位向量（Reset_Handler），之后按启动文件里的流程执行：

1. 取 SP：把向量表第一项加载到栈指针；
2. 跳转 Reset_Handler：向量表第二项即复位处理函数入口；
3. 拷贝.data：把已初始化全局变量的初值从 Flash 拷到 RAM；
4. 清零.bss：把未初始化全局变量清零；
5. SystemInit()：配置时钟（HSE/PLL，切到 168 MHz）；
6. main()：进入你的应用代码。

对应本工程的启动文件 startup/startup_stm32f40_41xxx.s，其中 Reset_Handler 依次完成 3-5 步（第 6 步 bl main）。我们已在其中去掉了 newlib 相关的 __libc_init_array 调用。

9.2 内存映射与链接脚本

链接脚本（linker/STM32F407VET6.ld）规定了各段放在哪里：

```
1 MEMORY
2 {
3     FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 512K    # 代码 + 只读数据
4     RAM   (xrw) : ORIGIN = 0x20000000, LENGTH = 128K    # 变量 + 栈 + 堆
5 }
```

- .text（代码）、.rodata（常量）→ FLASH；
- .data（已初始化全局变量，初值在 Flash，运行时拷到 RAM）；
- .bss（未初始化变量，运行时清零）→ RAM；
- 栈和堆在 RAM 末尾，脚本里定义了 _Min_Heap_Size / _Min_Stack_Size。

烧录.bin 时从 0x08000000 开始写 Flash。

9.3 向量表

启动文件里的向量表是一段地址列表，顺序与 ARM 规定一致：

```

1  .word  _estack           ; 0: 栈顶
2  .word  Reset_Handler    ; 1: 复位
3  .word  NMI_Handler      ; 2: 不可屏蔽中断
4  .word  HardFault_Handler ; 3: 硬错误
5  ...                     ; 之后是各外设中断

```

系统发生中断时，硬件按中断号查这张表，跳到对应处理函数。本工程把向量表放在 Flash 起始处（.isr_vector 段）。

9.4 Cortex-M4 与 FPU

STM32F407 是 Cortex-M4F，带单精度 FPU：

- 编译选项 `-mfpv4-sp-d16`：启用单精度 FPU 指令；
- `-mfloat-abi=hard`：浮点参数用 FPU 寄存器传递（更快）；
- 硬浮点 ABI 要求编译和链接使用相同选项，否则链接报 `float ABI mismatch` 类错误；
- `SystemInit()` 里会打开 FPU（设置 CPACR 寄存器），否则执行浮点指令会触发 `UsageFault`。

9.5 时钟树

`Core/src/system_stm32f4xx.c` 里的 `SetSysClock()` 负责：

1. 启用 HSE（外部晶振，本板 8 MHz）；
2. 配置 PLL 倍频到 168 MHz（SYSCLK）；
3. 配置 AHB/APB1/APB2 分频（APB1 最高 42 MHz，APB2 最高 84 MHz）；
4. 切换系统时钟源到 PLL，并等待切换完成（`RCC_CFGR_SWS` 忙等）。

外设时钟由 RCC 使能（例如 `RCC_AHB1PeriphClockCmd` 打开 GPIOD 的时钟）之后，外设寄存器才能被正常访问。

9.6 烧录与调试

`.bin/.hex` 生成后需要烧到芯片，常用工具：

- **ST-Link + STM32CubeProgrammer**（官方图形/命令行工具）；
- **OpenOCD**: `openocd -f interface/stlink.cfg -f target/stm32f4x.cfg -c "program build/MAKE.elf verify reset exit"`；
- **st-flash**（仅 ST-Link）: `st-flash write build/MAKE.bin 0x08000000`。

调试建议配合 GDB 与 OpenOCD 的 GDB server，或用 STM32CubeIDE 的调试器加载 `build/MAKE.elf`。

10 VS Code 开发与烧录

本章记录把本工程搬进 VS Code 之后「编译—烧录—调试」的完整配置，涉及四个文件：构建预设 CMakePresets.json、任务 tasks.json、烧录配置 openocd.cfg 与快捷键 keybindings.json。所有编译产物输出到 build/arm/，烧录地址为 Flash 起始 0x08000000。

10.1 总体工作流

1. 配置：cmake --preset arm，按预设生成 Ninja 工程；
2. 构建：cmake --build --preset arm，产出 build/arm/MAKE.elf 以及 MAKE.bin、MAKE.hex；
3. 烧录：OpenOCD按 openocd.cfg 把 MAKE.hex 写入 Flash，校验后复位运行。

这三步既能在 VS Code 里一键触发，也能在终端直接敲等价命令（见下文「常用终端命令」）。下面分别说明各配置文件。

10.2 构建预设 CMakePresets.json

预设把「生成器 + 工具链 + 输出目录」打包成一个名字 arm，之后配置只需一行 cmake --preset arm：

```
1 {
2   "version": 3,
3   "configurePresets": [
4     {
5       "name": "arm",
6       "generator": "Ninja",
7       "toolchainFile": "${sourceDir}/cmake/arm-none-eabi-gcc.cmake",
8       "binaryDir": "${sourceDir}/build/arm"
9     }
10  ],
11  "buildPresets": [
12    { "name": "arm", "configurePreset": "arm" }
13  ]
14 }
```

要点：binaryDir 决定产物目录，本工程统一放在 build/arm/，烧录命令与它保持一致即可。

10.3 VS Code 任务 tasks.json

任务（task）本质是「shell 命令的壳」：把要执行的命令、参数、分组打包成一个可反复触发的条目。本工程定义四个任务：

```
1 {
2   "version": "2.0.0",
3   "tasks": [
4     {
5       "label": "Build (arm)",
```

```
6     "type": "shell",
7     "command": "cmake --build --preset arm",
8     "group": "build",
9     "problemMatcher": []
10  },
11  {
12     "label": "Clean & Rebuild (arm)",
13     "type": "shell",
14     "command": "Remove-Item -Recurse -Force build/arm; cmake --preset arm; cmake --
15 build --preset arm",
16     "group": "build",
17     "problemMatcher": []
18  },
19  {
20     "label": "Flash Download (OpenOCD)",
21     "type": "shell",
22     "command": "openocd",
23     "args": [
24         "-f", "openocd.cfg",
25         "-c", "program build/arm/MAKE.hex verify reset exit"
26     ],
27     "group": "build",
28     "problemMatcher": []
29  },
30  {
31     "label": "Build & Download",
32     "dependsOn": ["Build (arm)", "Flash Download (OpenOCD)"],
33     "dependsOrder": "sequence",
34     "group": { "kind": "build", "isDefault": true },
35     "problemMatcher": []
36  }
37 }
```

几个容易踩的坑：

- `label` 是任务的唯一名字，`dependsOn` 按 `label` 精确匹配、区分大小写。写错名字不会报编译错误，而是点按钮时提示「找不到任务」。
- 一个任务只能有一个 `command`。想跑多条命令，要么用分号；连成一句（PowerShell），要么拆成多个任务再用 `dependsOn` 串联。
- `"group": { "kind": "build", "isDefault": true }` 把它设为默认构建任务，按 `Ctrl+Shift+B` 直接执行，不用再选。
- `problemMatcher` 置空数组，避免 VS Code 把终端输出误判成编译错误。

触发方式不止快捷键：Ctrl+Shift+P 输入 Tasks: Run Task 可从列表选；终端面板右上角的下拉菜单里也有 Run Task...

10.4 快捷键绑定 keybindings.json

把任务绑到专属快捷键(Ctrl+Shift+P 输入 Preferences: Open Keyboard Shortcuts (JSON)):

```
1 // Place your key bindings in this file to override the defaults
2 [
3   {
4     "key": "ctrl+alt+F",
5     "command": "workbench.action.tasks.runTask",
6     "args": "Clean & Rebuild (arm)"
7   },
8   {
9     "key": "ctrl+alt+O",
10    "command": "workbench.action.tasks.runTask",
11    "args": "Flash Download (OpenOCD)"
12  }
13 ]
```

注意：args 里的名字必须与 tasks.json 的 label 完全一致。

10.5 烧录配置 openocd.cfg

本工程的 openocd.cfg 内容如下 (Windows 下需先装好 ST-Link 驱动，并把 openocd 加入 PATH):

```
1 # Use ST-Link adapter and STM32F4 target
2 source [find interface/stlink.cfg]
3 source [find target/stm32f4x.cfg]
4
5 adapter speed 240
6 stm32f4x.cpu configure -event reset-init { adapter speed 240 }
7 stm32f4x.cpu configure -event reset-start { adapter speed 240 }
```

各行的作用：

- source [find ...]: 加载 ST-Link 调试器和 STM32F4 目标的官方配置。新版用 interface/stlink.cfg, 旧名 stlink-v2.cfg 已废弃 (会有告警)。
- adapter speed 240: 把 SWD 时钟设成 240 kHz。
- 最后两行覆盖 stm32f4x.cfg 自带的 reset-init / reset-start 事件处理器。不覆盖的话, OpenOCD 会在复位/烧录时自动把速度顶到 2000/8000 kHz (ST-Link V2 达不到, 被迫降档)。

10.5.1 ST-Link 接线

- SWDIO → 芯片 PA13;

- **SWCLK** → 芯片 PA14;
- **GND** → GND, **VCC(3V3)** → 3.3V (板子已自供电也可不接);
- **RESET (NRST)** 为可选项: 不接时删掉 `reset_config` 相关行, OpenOCD 走软件复位 (向内核寄存器写 `SYSRESETREQ`), 同样能烧录复位。接了 NRST 想更稳才用 `reset_config srst_only`。

10.5.2 关于 SWD 速度档位

ST-Link 的 SWD 速度是离散档位 (如 240/360/480/1800/4000 kHz), 不是任意值。请求一个不存在的档位时, OpenOCD 会打印 `Unable to match requested speed X, using Y` 并自动降档——这只是降速提示, 不是错误。直接设成 240 (ST-Link 支持的档位) 就不会出现该提示, 也是最稳的低速选择。

10.6 常用终端命令

任务的底层就是下面这些命令, 在任意终端都能直接敲 (Windows 默认用 PowerShell):

```
1 # 构建 (增量, CMake 会自动重新配置)
2 cmake --build --preset arm
3
4 # 清理 + 配置 + 全量构建
5 Remove-Item -Recurse -Force build/arm; cmake --preset arm; cmake --build --preset arm
6
7 # 烧录 (下载 + 校验 + 复位运行)
8 openocd -f openocd.cfg -c "program build/arm/MAKE.hex verify reset exit"
9
10 # 构建 + 烧录一次完成
11 cmake --build --preset arm; openocd -f openocd.cfg -c "program build/arm/MAKE.hex
    verify reset exit"
```

若终端是 `cmd` (非 PowerShell), 把 `Remove-Item -Recurse -Force` 换成 `rmdir /s /q`, 且路径反斜杠 `build\arm`。

10.7 在线调试 (可选)

想单步、看变量, 可安装扩展 **Cortex-Debug** (ID `marus25.cortex-debug`), 然后新建 `.vscode/launch.json`:

```
1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "name": "Flash & Debug (OpenOCD)",
6       "type": "cortex-debug",
7       "request": "launch",
8       "servertype": "openocd",
9       "cwd": "${workspaceFolder}",
```

```

10     "executable": "${workspaceFolder}/build/arm/MAKE",
11     "configFiles": ["${workspaceFolder}/openocd.cfg"],
12     "runToEntryPoint": "main"
13 }
14 ]
15 }

```

按 F5 即自动启动 OpenOCD、下载固件并停在 main()。前提是固件带调试符号:当前 CMakeLists.txt 未加 -g, 产物没有 .debug 段, 若要源码级调试, 需在 target_compile_options 里补上 -g 再重新构建。

11 存储器、内存管理与 OTA 升级

本章从整个地址空间讲起, 逐步落到本工程的 IAP 分区、链接脚本、上电流程和 OTA 升级业务。读完你会明白: 为什么程序烧在 0x08000000、为什么跳转标志放在 0x08060000、为什么上电后程序知道自己该做什么。

11.1 Cortex-M4 的 4GB 地址空间

Cortex-M 内核采用**统一编址**: 代码、内存、外设、寄存器全部映射到同一个 4GB 地址空间里, CPU 访问它们的方式完全一样。整个空间按用途分成六大块:

地址范围	大小	用途
0x00000000 – 0x1FFFFFFF	512MB	代码区 (Flash / ROM)
0x20000000 – 0x3FFFFFFF	512MB	SRAM (变量、栈、堆)
0x40000000 – 0x5FFFFFFF	512MB	片上外设 (GPIO、串口、定时器等)
0x60000000 – 0x9FFFFFFF	1GB	外部 RAM
0xA0000000 – 0xDFFFFFFF	1GB	外部设备
0xE0000000 – 0xFFFFFFFF	512MB	内核系统区 (NVIC、SysTick、SCB 等)

MCU 上电后第一条指令、中断处理、外设寄存器读写, 全都是在这张表里按地址干活。

11.2 STM32F407 的实际内存地图

把上面的通用框架填上 STM32F407 的具体资源 (本工程芯片 512KB Flash、128KB SRAM):

地址范围	大小	用途
0x00000000 – 0x0007FFFF	512KB	Flash 别名 (BOOT0=0 时映射到主 Flash)
0x08000000 – 0x0807FFFF	512KB	主 Flash: 代码 + 常量 (.text/.rodata)
0x1FFF0000 – 0x1FFF77FF	30KB	系统存储器: 出厂 Bootloader (ISP)
0x1FFFC000 – 0x1FFFC00F	16B	选项字节: 读保护、写保护等配置
0x10000000 – 0x1000FFFF	64KB	CCMRAM: CPU 专用 RAM (DMA 不能访问)
0x20000000 – 0x2001FFFF	128KB	主 SRAM: 变量、栈、堆
0x40000000 – 0x5FFFFFFF	–	外设区: AHB/APB 上的全部外设寄存器
0xE000ED08	4B	SCB->VTOR: 向量表偏移寄存器 (跳转的关键)
0xE000E010	4B	SysTick 定时器

BOOT0 引脚的三种启动方式（决定 CPU 从哪取向量表）：

1. **BOOT0=0**：从主 **Flash**（0x08000000）启动——正常运行；
2. **BOOT0=1, BOOT1=0**：从系统存储器（出厂 Bootloader）启动，用于串口 ISP 烧录；
3. **BOOT0=1, BOOT1=1**：从 **SRAM** 启动，调试用。

本工程默认 **BOOT0=0**，所以上电永远从主 Flash 的 0x08000000 开始。

11.3 Flash 物理特性与扇区管理

Flash 有两个关键物理约束，所有分区设计都从这两条出发：

- 只能把 1 写成 0：已编程的位只能 1→0，想变回 1 只能擦除；
- 擦除的最小单位是扇区：不能只擦一个字节。

STM32F407的 512KB Flash 分 8 个扇区（S0–S7），大小不相等：

扇区	地址	大小
S0	0x08000000	16KB
S1	0x08004000	16KB
S2	0x08008000	16KB
S3	0x0800C000	16KB
S4	0x08010000	64KB
S5	0x08020000	128KB
S6	0x08040000	128KB
S7	0x08060000	128KB

由此得出三条设计准则：分区边界必须对齐扇区（否则擦 A 区会误擦 B 区）；代码区之间要留够擦除粒度；需要反复改写的的数据（如跳转标志）要独占一个扇区。

11.4 本工程的 IAP 分区

本工程把 512KB Flash 划分如下（即方案甲：Boot + 双 App）：

区	扇区	地址	大小
Bootloader	S0 + S1	0x08000000	32KB
App1	S2 + S3	0x08008000	32KB
App2	S4	0x08010000	64KB
备用	S5 + S6	0x08020000	256KB
跳转标志区	S7	0x08060000	128KB

为什么 **Boot** 必须在 **0x08000000**：BOOT0=0 时 CPU 上电只从 0x08000000 取向量表，所以这个地址必须是“第一个跑起来的程序”。

为什么跳转标志放在 **0x08060000**（你问的核心）：三个理由——

1. **独立成扇**：改标志要先整扇擦除，如果标志和代码同扇区，一擦就把代码擦没了；S7只存标志，擦它不伤任何程序；
2. **远离代码区**：S7是最后一个扇区，Boot/App 升级、写入时永远不会碰到它；
3. **空间充裕**：整个 128KB 只存 4 字节标志虽然”浪费”，但换来绝对安全，而且剩下的空间还能存升级参数、日志。

所以烧录时写 0x08060000 就是写 S7 首地址——这一步在「切换 App」时用 flash write_bank 完成。

11.5 链接脚本与 RAM 布局

程序编译后按段（section）存放，由链接脚本（.ld）决定每段放哪。本工程 linker/*.ld 的核心：

```
1 FLASH (rx)      : ORIGIN = 0x08008000, LENGTH = 32K    ; App1 示例
2 RAM   (xrw)     : ORIGIN = 0x20000000, LENGTH = 128K
3 _estack = 0x20020000; ; RAM 顶部 = 初始栈指针
```

各段的作用：

- **.isr_vector**：向量表，永远放 Flash 最前面；
- **.text**：代码，放 Flash；
- **.rodata**：只读常量（字符串、查表数据），放 Flash；
- **.data**：已初始化全局变量，初值暂存 Flash，上电时拷到 RAM；
- **.bss**：未初始化/清零的全局变量，直接放 RAM，上电清零；
- **堆 + 栈**：放在 RAM 末尾（_Min_Heap_Size / _Min_Stack_Size 定义大小）。

RAM 一共 128KB（0x20000000–0x2001FFFF），栈顶 0x20020000 就是整个 RAM 的最高地址。App 和 Boot 各自独立使用这份 RAM（同一时刻只有一个在跑，不会冲突）。

11.6 上电启动流程

从按复位键到 main()，CPU 依次完成：

1. 读 0x00000000（映射到 0x08000000）→ 存入 SP；
2. 读 0x00000004 → 得到 Reset_Handler 入口；
3. 执行启动文件 startup_stm32f40_41xxx.s：
 - (a) 把 .data 初值从 Flash 拷到 RAM；
 - (b) 清零 .bss；
 - (c) 调用 SystemInit()（配置时钟 PLL）；

4. 跳进 main()。

在本工程 Boot 里，main() 只做一件事——看标志、选 App：

```

1  int main(void)
2  {
3      SystemInit();
4      uart_init();
5      switch (iap_read_flag()) {      // 读 0x08060000
6          case FLAG_UPGRADE: iap_upgrade();      break;
7          case FLAG_RUN_APP2: iap_load_app(APP2_ADDR); break;
8          default:          iap_load_app(APP1_ADDR); break;
9      }
10     while (1) { }
11 }

```

11.7 跳转机制：向量表与 VTOR

每个程序自带一张**向量表**：第 0 项是栈顶，第 1 项是复位入口，后面是各中断服务函数地址。CPU 发生中断时，就按“当前向量表的位置”查表。

App 被链接到别处后（如 App1 在 0x08008000），它的向量表也在那里。但 CPU 默认向量表还在 0x08000000（Boot 区）——所以跳转前必须把 SCB->VTOR 拨到 App 基址，否则一来中断就查错表、直接死机。这就是 iap_load_app 的完整逻辑：

```

1  void iap_load_app(uint32_t appxaddr)
2  {
3      uint32_t sp      = *(volatile uint32_t *)appxaddr;
4      uint32_t reset = *(volatile uint32_t *)(appxaddr + 4);
5
6      if ((sp >= 0x20000000u) && (sp <= 0x20020000u)) { // 栈顶在 RAM 范围
7          __disable_irq();
8          SCB->VTOR = appxaddr;      // 向量表拨过去
9          __set_MSP(sp);              // 换栈
10         ((iapfun)reset)();          // 跳到 App 的 Reset_Handler
11     }
12 }

```

- 0x20000000 / 0x20020000 分别是 RAM 的起止边界，加 u 后缀表示无符号整数（避免与 uint32_t 比较的告警）；
- 用 <= 而不是 <：因为本工程栈顶恰好等于 RAM 顶 0x20020000，< 会把合法的栈顶误判为非法；
- 跳之前关中断，防止切换瞬间有中断闯进来查旧向量表。

App 自己的 main() 第一行也要写 SCB->VTOR = 自己的基址，保证中断路由正确。

11.8 OTA 升级业务设计

11.8.1 跳转标志（三态）

标志存在 S7 首地址 0x08060000，4 字节：

值	含义	何时写入
0xFFFFFFFF	出厂默认	Flash擦除后天然如此
0x0000AAAA	跳 App1	升级 App1 成功后 Boot 写
0x00005555	跳 App2	升级 App2 成功后 Boot 写
0x00003333	进升级模式	App 想升级时先写此值再复位

为什么三态而不是两态：擦除后天然 0xFFFFFFFF，表示“回默认”，即使标志意外被擦掉，系统也有确定行为，不会变砖。

11.8.2 升级协议与校验

上位机通过串口发一帧：

```
1 [0xAA][0x55][target:1B][size:2B 小端][bin 数据...][crc16:2B 小端]
2 target: 1=App1, 2=App2
3 crc16: 只对 bin 数据计算 (CCITT 0x1021, 初值 0xFFFF)
```

Boot 收帧流程：等帧头 → 收目标区号 → 收大小 → 收数据 → 收 CRC → 计算 CRC 比对 → 通过后擦目标扇区、逐字写入 → 写跳转标志 → 复位。

为什么需要「大小 + CRC 双重校验」：串口会丢字节、错位。只按长度收满就写，收到坏 bin 就变砖；CRC 能查出内容是否完好，两个都过了才允许写 flash。

11.8.3 双区 A/B 为什么能不停机

App1 在跑的时候，Boot 往 **App2**（S4）写新固件——两个区互不干扰。写完复位切换，停机时间只有复位那一瞬。App2 在跑时同理可写 App1。对比单区方案：升级时必须退回 Boot 等收完数据，系统全程停机。

11.8.4 防砖设计

1. 先擦目标扇区，再写，再置标志：任何时刻掉电，标志都指向“可用的旧区”，不会跳到写了一半的新区；
2. 写后读回验证：写完比对，不一致就不置跳转标志；
3. App 启动自检：App 校验自己关键数据失败时，跳回另一个区，天然回退到旧版本；
4. Boot 永不更新：只要 Boot 在，随时能进升级模式救砖。

11.9 完整升级时序（串联本章所有概念）

1. App1 运行中收到升级指令 → 写 0x3333 标志 → 复位；
2. Boot 启动，读标志 = 0x3333 → 进入 iap_upgrade();

3. Boot 串口收帧，CRC 校验通过；
4. 擦除 S4 (App2 所在扇区) → 写入新固件 → 读回验证；
5. 写 0x5555 (跳 App2) → 复位；
6. Boot 再启动，读 0x5555 → SCB->VTOR=0x08010000，跳转 App2，PE4 LED 闪烁——升级完成。

升级链路不局限于串口：帧格式与校验逻辑可以原样复用到 CAN、无线数传等任何通信链路（本工程后续可结合 MAVLink 业务把升级帧挂到数传上）。